

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I M Kavya 192472370(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: kavya

Reverse Engineering

1. Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

2. Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

4. Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

ANNEXURE A – REVERSE ENGINEERING

Course: Artificial Intelligence (CSA17)

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.

Question 1: Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations, identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

1. System Overview and Data Sources

A large e-commerce recommendation engine continuously ingests heterogeneous data streams: clickstream logs, page views, add-to-cart events, purchase history, product catalog metadata, search queries, and user ratings/reviews. These events are captured through event-tracking SDKs embedded in the web and mobile applications, then pushed into a distributed message broker such as Apache Kafka, which acts as the ingestion buffer between the front-end application layer and the back-end analytics layer. Reverse engineering this pipeline reveals that the system must decouple fast-changing user behavior from the heavier batch computations needed for model training, which is why a message queue sits at the very front of the architecture.

2. Data Processing Using PySpark and RDDs

Once ingested, raw event data is loaded into Apache Spark as Resilient Distributed Datasets (RDDs), which allow the platform to process massive, partitioned datasets in parallel across a cluster of worker nodes. The typical transformation pipeline can be reconstructed as follows:

- Extraction: `rawEventsRDD = sc.textFile("kafka_topic_dump")` loads the raw JSON/CSV event logs into an RDD distributed across partitions.
- Cleaning and Filtering: `rawEventsRDD.filter(lambda x: isValidEvent(x))` removes malformed, duplicate, or bot-generated records.
- Mapping: `parsedRDD = filteredRDD.map(lambda x: parseEvent(x))` converts each raw record into a structured tuple of (userId, itemId, eventType, timestamp).
- Aggregation: `userItemRDD.reduceByKey(lambda a, b: a + b)` aggregates interaction weights (views = 1, cart = 3, purchase = 5) for every (user, item) pair to build an implicit-feedback interaction matrix.

- Joins: `interactionRDD.join(productCatalogRDD)` enriches each interaction with product attributes such as category, price, and brand, enabling content-based features alongside collaborative ones.

This RDD-based pipeline is chosen over simple single-machine processing because RDDs offer fault tolerance through lineage graphs (if a partition is lost, Spark recomputes it from the DAG rather than restarting the whole job) and in-memory caching (`interactionRDD.cache()`) that dramatically accelerates iterative machine-learning algorithms such as Alternating Least Squares (ALS) for collaborative filtering.

3. Model Training and Recommendation Generation

The transformed interaction RDD is fed into Spark MLlib's ALS algorithm, which factorizes the sparse user-item interaction matrix into latent user and item feature vectors. This training job runs as a batch process, typically scheduled hourly or daily, and produces a model that can be queried to generate top-N recommendations for each user. For real-time responsiveness, a lighter-weight streaming layer (Spark Structured Streaming or a similar micro-batch engine) recalculates short-term signals — trending items, session-based similarity — so that recommendations reflect what the user just did, not only historical behavior.

4. Role of Intelligent Agents

Reverse engineering the dynamic update behavior of the system suggests the presence of intelligent software agents operating continuously between the model and the user interface:

- Monitoring Agents observe live user interactions and detect meaningful behavior shifts (e.g., a sudden interest in a new category).
- Decision Agents select which candidate recommendations to surface, balancing exploration (introducing novel items) against exploitation (showing known high-affinity items), often using a multi-armed bandit strategy.
- Feedback Agents close the loop by logging whether a recommendation was clicked or ignored, feeding this signal back into the training pipeline to continuously refine the model — an architecture consistent with a perception-action-learning cycle typical of intelligent agents.

Overall, the reverse-engineered architecture is a hybrid batch-plus-streaming system: PySpark and RDDs provide scalable, fault-tolerant offline model training, while a layer of intelligent agents provides the real-time adaptivity that makes recommendations feel instantaneous and personalized.

Question 2: Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

1. Data Sources and Streaming Ingestion

A smart traffic management system draws data from IoT sensors embedded in roads (inductive loop detectors), traffic cameras with computer-vision vehicle counting, GPS data from connected vehicles and ride-hailing apps, and citizen-reported congestion via mapping applications. Because traffic conditions change second by second, this data cannot wait for batch processing; instead it is streamed continuously into the system using a framework such as Spark Structured Streaming, which internally represents incoming micro-batches as sequences of RDDs (DStreams).

2. RDD-Based Stream Processing Pipeline

- Ingestion: `sensorStream = ssc.socketTextStream(host, port)` or a Kafka-based DStream continuously receives sensor readings every few seconds.
- Windowed Aggregation: `sensorStream.window(windowLength=60, slideInterval=10)` groups readings into rolling 60-second windows updated every 10 seconds, smoothing out sensor noise while remaining responsive.
- Transformation: `windowedRDD.map(lambda r: (intersectionId, vehicleCount)).reduceByKey(lambda a,b: a+b)` computes per-intersection vehicle density in near real time.
- Congestion Scoring: `densityRDD.mapValues(lambda count: computeCongestionScore(count))` converts raw counts into a normalized congestion index per intersection, factoring in road capacity and historical baselines.
- Fault Tolerance: because RDD lineage is preserved even across streaming micro-batches, a lost worker node can recompute only the affected partition rather than losing the whole traffic snapshot — essential for a system that cannot afford downtime.

3. Agent Interactions and Real-Time Decision Making

Reverse engineering the control logic reveals a multi-agent architecture operating at the level of individual intersections and coordinating regionally:

- Local Signal-Control Agents at each intersection consume the congestion index for their junction and decide the optimal green-light duration for each approach, typically using a reinforcement-learning policy trained to minimize average wait time.

- Coordination Agents at the corridor or district level communicate with neighboring intersection agents (often via a publish-subscribe channel) to implement 'green wave' synchronization, so vehicles moving along a major road hit consecutive green lights.
- Emergency-Override Agents can pre-empt normal signal logic when priority vehicles (ambulances, fire trucks) are detected, temporarily overriding the optimization policy for safety.

4. Operating Under Time Constraints

Because a traffic signal decision that arrives even a few seconds late is useless, the system is architected for low-latency decisions: lightweight in-memory RDD operations, pre-trained models that only require fast inference (not retraining) at decision time, and edge-level or near-edge computation to avoid round-trip latency to a distant data center. Heavier tasks — retraining the reinforcement-learning policy on the day's full traffic patterns — are pushed to an offline batch layer that runs overnight, so the real-time path always stays fast while the underlying models still improve over time. This split between a fast streaming/agent decision loop and a slower batch learning loop is the defining architectural pattern reverse-engineered from the system's real-time behavior.

Question 3: Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

1. Behavioral Observations Driving the Reverse-Engineered Design

The observable fact that fraud alerts appear within milliseconds of a transaction implies that the core scoring model cannot be a heavy batch job; it must be a pre-trained model served through a low-latency streaming pipeline, with only historical model refinement happening offline. This single observation constrains the entire architecture that follows.

2. Data Pipeline and Feature Extraction Using PySpark RDDs

- Ingestion: every card swipe, online payment, or transfer request is published to a streaming layer (Kafka) and consumed as micro-batch RDDs: `txnRDD = kafkaStream.map(lambda x: parseTransaction(x))`.
- Feature Extraction: `txnRDD.map(lambda t: extractFeatures(t))` derives features such as transaction amount, merchant category, geolocation distance from the user's last known location, time since the previous transaction, device fingerprint, and deviation from the user's historical spending pattern.

- Historical Join: `featureRDD.join(userProfileRDD)` enriches each transaction with a rolling user profile (average spend, typical merchants, home location) maintained in a fast key-value store and periodically refreshed by a Spark batch job.
- Aggregated Risk Signals: `featureRDD.reduceByKey(lambda a,b: combineRiskFactors(a,b))` can combine multiple risk signals per account within a short time window, catching patterns like rapid repeated small transactions ('card testing').

3. Anomaly Detection Layer

The engineered feature vector is passed to a pre-trained anomaly detection model — commonly an ensemble of a supervised classifier (gradient-boosted trees or logistic regression trained on labeled historical fraud cases) and an unsupervised outlier detector (isolation forest or autoencoder reconstruction error) to also catch previously unseen fraud patterns. The model outputs a fraud probability score; RDD partitioning ensures that scoring for thousands of concurrent transactions is distributed across the cluster and executed in parallel rather than sequentially, which is what allows the millisecond-level response time observed externally.

4. Agent-Based Alert Mechanisms

- Scoring Agents evaluate each transaction's risk score the instant it is computed and apply a decision threshold (approve, flag for review, or block).
- Escalation Agents route high-risk transactions to a case-management queue for human fraud analysts, attaching the contributing feature evidence so the review is fast and explainable.
- Notification Agents trigger real-time SMS, push, or app notifications to the cardholder, and can require step-up authentication (OTP) before the transaction is allowed to proceed.
- Learning Agents feed analyst decisions (confirmed fraud vs. false positive) back into the labeled training set, so the offline PySpark batch retraining job continuously improves model accuracy — closing the loop between real-time inference and periodic model updates.

The reconstructed architecture therefore separates a fast, RDD-parallelized real-time scoring path from a slower PySpark batch layer that retrains the anomaly-detection models, with intelligent agents bridging both layers through scoring, escalation, notification, and feedback.

Question 4: Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed

using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

1. Learner Data Collection

The platform continuously logs granular interaction data: quiz attempt results and time taken, video watch duration and replay/skip behavior, exercise completion rates, forum participation, and login frequency. This data is collected per learner, per learning module, and streamed or batched into the analytics backend for processing.

2. RDD-Based Transformation Pipeline

- Extraction: `learnerLogsRDD = sc.textFile("activity_logs/")` loads raw activity logs distributed across the cluster.
- Parsing and Structuring: `parsedRDD = learnerLogsRDD.map(lambda x: parseActivity(x))` converts logs into structured tuples of (studentId, moduleId, score, timeSpent, attempts).
- Per-Student Aggregation: `parsedRDD.map(lambda x: (x.studentId, x)).groupByKey()` groups all activity by learner, enabling computation of a rolling mastery estimate per topic.
- Performance Scoring: `studentGroupedRDD.mapValues(lambda records: computeMasteryScore(records))` calculates a proficiency metric per topic, often using an Item Response Theory or Bayesian Knowledge Tracing style update rule.
- Engagement Metrics: `parsedRDD.filter(lambda x: x.eventType == 'video').reduceByKey(...)` separately tracks engagement signals (completion rate, drop-off points) which feed into a distinct engagement score used alongside performance.

RDD transformations are well suited here because student activity data is naturally large-scale (many learners, many events each) and because the lineage-based fault tolerance of RDDs ensures that a node failure during a nightly batch recomputation of mastery scores does not corrupt or lose partial results.

3. Role of Intelligent Agents in Adaptive Learning

- Diagnostic Agents continuously interpret the mastery and engagement scores to classify a learner's current state (struggling, on-track, or ready to advance) on a per-topic basis.
- Content-Selection Agents use this diagnosis to dynamically choose the next piece of content — an easier remedial exercise, an equivalent practice item, or a more advanced challenge — implementing the adaptive difficulty the platform is known for.
- Feedback Agents generate real-time hints, explanations, or encouragement messages tailored to the specific error pattern a student exhibits, rather than generic feedback.

- Engagement-Monitoring Agents detect early signs of disengagement (long inactivity, repeated failed attempts) and trigger interventions such as reminder notifications or simplified content to prevent drop-off.

Reverse engineering the system's adaptivity therefore reveals a closed feedback loop: RDD-based batch and near-real-time transformations continuously update a per-learner knowledge model, and a layer of intelligent agents translates that model into concrete, personalized pedagogical actions — content sequencing, feedback, and intervention — which in turn generates new activity data that refines the model further.

Question 5: Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

1. Log and Metric Collection at Cloud Scale

A cloud monitoring platform continuously collects system-level metrics (CPU, memory, disk I/O, network throughput), application logs, and service health checks from potentially thousands of virtual machines, containers, and microservices. Given this volume, logs are shipped through a distributed collection agent (such as Fluentd or a custom log-forwarder) into a message queue and from there into a Spark cluster for processing.

2. PySpark Processing of Large-Scale Logs

- Ingestion: `logsRDD = sc.textFile("s3://cloud-logs/*")` loads log files spread across distributed storage, automatically partitioned for parallel processing.
- Parsing: `parsedRDD = logsRDD.map(lambda line: parseLogLine(line))` extracts structured fields such as timestamp, service name, host, response time, and error codes from unstructured log text.
- Filtering: `errorRDD = parsedRDD.filter(lambda log: log.level == 'ERROR' or log.responseTime > threshold)` isolates records indicating potential problems.
- Aggregation by Service: `parsedRDD.map(lambda x: (x.serviceName, x.responseTime)).groupByKey().mapValues(computeStats)` computes rolling statistics (mean, p95, p99 latency) per service, which forms the baseline against which anomalies are detected.

- PySpark's ability to distribute this map-filter-reduce pipeline across a large cluster is what allows the platform to process logs from thousands of nodes without a single-machine bottleneck, which is the key capability the reverse-engineered design must explain.

3. RDD Operations Supporting Fault Tolerance

RDDs track their transformation history as a lineage graph (a Directed Acyclic Graph of operations) rather than persisting every intermediate result. If a worker node processing part of the log data fails, Spark's driver simply recomputes the lost partition from its lineage using the original data source, rather than restarting the entire job. This is critical for a monitoring system, since the monitoring pipeline itself must remain reliable even while the infrastructure it is observing is experiencing failures. Additionally, caching frequently reused RDDs (e.g., recent baseline statistics with `.persist()`) in memory across the cluster reduces recomputation cost for the anomaly-scoring step that runs on every new batch of logs.

4. Intelligent Agents Coordinating Monitoring, Scaling, and Response

- Monitoring Agents continuously compare incoming metrics against the learned baseline (from the aggregation step above) and flag statistically significant deviations as candidate anomalies.
- Diagnosis Agents correlate anomalies across related services (e.g., a spike in database latency coinciding with an increase in application errors) to distinguish a root cause from a downstream symptom.
- Auto-Scaling Agents interpret sustained resource-utilization trends (not just momentary spikes) and issue scale-out or scale-in commands to the cloud orchestrator (such as Kubernetes' Horizontal Pod Autoscaler), balancing responsiveness against the cost and instability of over-aggressive scaling.
- Response/Remediation Agents can automatically trigger predefined runbooks — restarting an unhealthy service instance, rerouting traffic away from a failing node, or paging an on-call engineer when automated remediation is not confidently applicable.

Taken together, the reverse-engineered system is a layered architecture: PySpark and RDDs provide the scalable, fault-tolerant log-processing backbone, while a coordinated set of intelligent agents converts the resulting metrics and anomaly signals into autonomous monitoring, scaling, and remediation actions with minimal human intervention.